

Module 5: Fault Tolerance

5.1 Fundamentals of Fault Tolerance

A distributed system is genuinely fault-tolerant when it can continue executing its operations seamlessly despite component hardware or software failures. It helps to separate three related terms: a **fault** is the underlying defect (a bug, a worn-out disk sector), an **error** is the incorrect internal state that fault causes, and a **failure** is when the system visibly stops behaving as specified.

Fault Type	Description
Transient	Occurs once and then disappears (e.g., a brief network blip).
Intermittent	Occurs, disappears, then recurs unpredictably (the hardest to diagnose).
Permanent	Persists until the faulty component is repaired or replaced (e.g., a burnt-out disk).

Systems are also described by the kind of **failure model** they defend against, ranging from mild to severe:

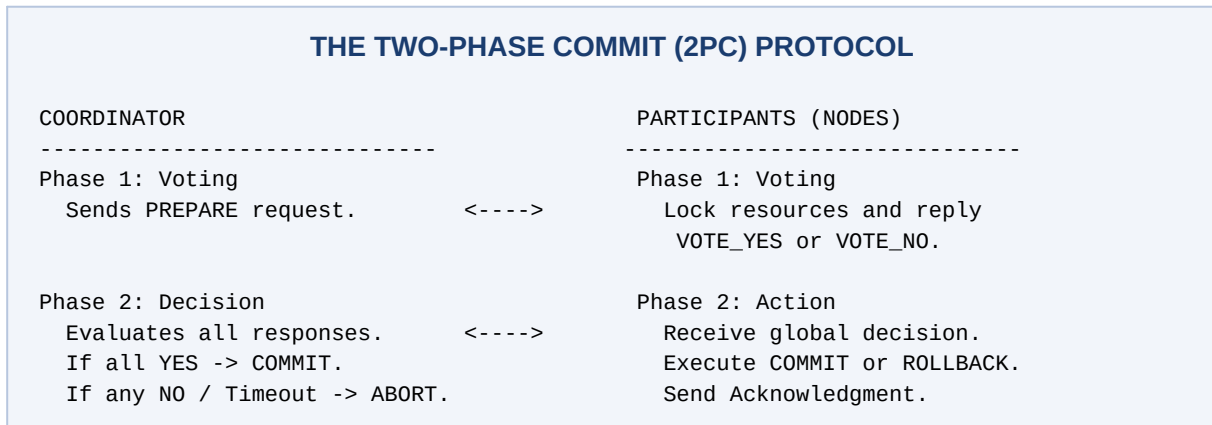
Failure Model	Behavior
Crash failure	A node simply stops; it does not send incorrect data, it just goes silent.
Omission failure	A node fails to send or receive some messages.
Timing failure	A node responds, but outside its agreed time bound.
Byzantine failure	A node behaves arbitrarily, possibly sending conflicting or malicious data — the hardest to tolerate.

■ The Off-Grid Solar Setup Analogy

Consider an off-grid solar power configuration consisting of solar panels, a charge controller, a central power inverter, and a bank of tubular batteries wired in parallel. If one single tubular battery degrades or a cell in a solar panel is permanently shaded, the system is designed to tolerate this fault. The inverter doesn't shut down the entire grid; instead, it dynamically draws the required load from the remaining healthy batteries. Distributed systems use redundancy and group communication in the exact same way to survive node failures.

5.2 Distributed Commit Protocols

When an operation requires changes across multiple nodes, the system must ensure either all nodes commit the change, or none do (this is Atomicity, from ACID, applied across machines).



Walking through it step by step: the coordinator first asks every participant "can you commit?" (PREPARE). Each participant locks the resources it needs and votes YES or NO. Only once the coordinator has heard back from everyone does it make a single global decision and broadcast it. This guarantees atomicity — but at a cost.

While 2PC is standard, it suffers from a fatal flaw: if the coordinator crashes during Phase 2, participants who already voted YES are **blocked indefinitely**, holding their locks and unable to safely decide whether to commit or abort on their own. **Three-Phase Commit (3PC)** introduces an intermediate "Pre-Commit" phase, so participants have enough information to make progress even if the coordinator disappears — though it adds an extra round of network latency.

For the most hostile environments — where nodes might not just crash but actively lie (Byzantine failures) — systems use **Byzantine Fault Tolerant (BFT)** consensus protocols, which require a supermajority of nodes to agree before any action is taken, so no small group of malicious or broken nodes can corrupt the outcome.

5.3 Recovery and Checkpointing

To survive failures, systems employ two broad recovery strategies:

- **Backward Recovery:** Saving regular snapshots or "checkpoints" of the system state and rolling back to the last good checkpoint upon failure. Simple, but you lose any work done since the last checkpoint.
- **Forward Recovery:** Using error-correcting codes or redundant computation to push forward to a new, stable state without rolling back — harder to design, but avoids losing progress.

Choosing how often to checkpoint is itself a trade-off: checkpointing too rarely means losing more work on failure, while checkpointing too often adds overhead that slows the system down during normal operation.

■ Key Takeaway

2PC guarantees atomicity but can block forever if the coordinator dies mid-decision; 3PC and BFT protocols exist specifically to remove that single point of failure, at the cost of extra messages and latency.

Glossary of Key Terms

Term	Meaning
Fault	The underlying defect that can lead to an error or failure.
Byzantine failure	A failure where a node behaves arbitrarily or maliciously.
2PC	Two-Phase Commit — a voting-then-deciding atomic commit protocol.
Checkpoint	A saved snapshot of system state used for recovery.

What You Should Know

- The classification of faults (transient, intermittent, permanent) and of failure models (crash, omission, timing, Byzantine).
- The exact step-by-step mechanism of the Two-Phase Commit protocol.
- Why 2PC can lead to blocking, how 3PC attempts to solve it, and why Byzantine environments need supermajority consensus.
- The concept of checkpointing for backward recovery, versus forward recovery, and the trade-off in checkpoint frequency.